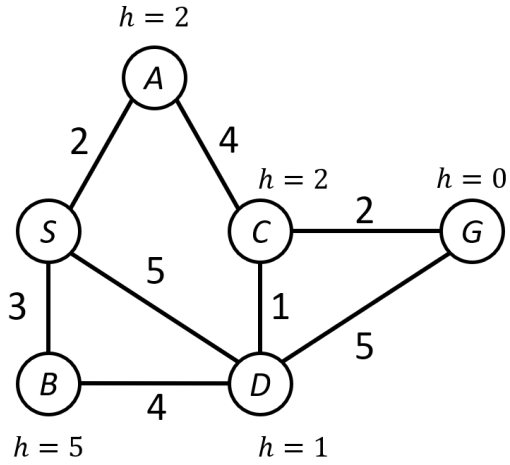
COMS W4701: Artificial Intelligence, Summer 2026

Homework 1

Instructions: Prepare submissions **only** for problems marked "Graded". Compile written responses in a single, typed PDF file. Coding solutions may be directly implemented in the provided Python file(s). Do not modify any filenames or code outside of the indicated sections. Submit all rquired files on Pensive in the appropriate assignment bins, and tag the pages of your PDF file. Please be mindful of the deadline, as well as our policies on citations and academic honesty.

[No Submission] Problem 1: Best-First Search

In the state space graph below, S is the start state and G is the goal state. Costs are shown along edges and heuristic values are shown adjacent to each node. All edges are undirected (or bidirectional). Assume that search algorithms expand nodes according to alphabetical order when ties are present or when considering nodes in the same tier of the search tree.



The *early* goal test refers to checking that a state is the goal right *before* insertion into the frontier. The *late* goal test refers to checking it right *after* popping from the frontier. *Multiple path pruning* prevents a state from being added to the frontier if it has already been reached, unless it is cheaper according to UCS or A* search.

1. Suppose we run depth-first search. Assuming that it finishes, list the sequence of states that are **popped** from the frontier, as well as the state sequence **solution**, for each of the following scenarios: a) early goal test with pruning, b) late goal test with pruning, and c) late goal test without pruning. Otherwise, indicate that DFS does not finish.

2. Repeat part 1 above for BFS.

3. Suppose we run UCS with multiple path pruning. List the sequence of popped states as well as the solution for each of the following scenarios: a) early goal test, b) late goal test.

4. Repeat part 3 above for A* search.

5. Suppose we change the heuristic function. Find the range of nonnegative heuristic values for $h(A)$ that would cause A* (utilizing a late goal test and reached table) to return an suboptimal solution, or indicate if A* would behave optimally regardless of $h(A)$. Repeat for $h(B)$, $h(C)$, and $h(D)$.

[No Submission] Problem 2: Local Search

We will use local search to solve the 4-queens problem. To simplify representation, a state will be represented by a set of 4 coordinate tuples, one for each queen. Following NumPy indexing, the top left cell has coordinates $(0, 0)$ and the bottom right cell has coordinates $(3, 3)$. So the example state shown below is $\{(0, 1), (1, 3), (2, 0), (3, 2)\}$.

We will define neighboring states as those where exactly one queen is moved to a different column. We will use the "min conflicts" evaluation function h, which counts the number of different queen pairs that are in the same row, column, or diagonal.

1. Consider the initial state with all queens in the left column, or $\{(0, 0), (1, 0), (2, 0), (3, 0)\}$. What is its h value? Remember to count *all* pairs of queens in conflict. Are there any neighboring states with the same or greater h value? What kind of feature would this state represent in the state space landscape?

2. Starting from the above initial state, trace the hill-"descent" procedure in which we repeatedly move to a neighboring state with the *lowest* h value among all neighbors, until we can improve no further. Write out the state and h value after each iteration.

3. Now consider the initial state $\{(0, 1), (1, 3), (2, 2), (3, 0)\}$. What is its h value? Are there any neighboring states with the same or lower h value? What kind of feature would this state represent in the state space landscape?

4. What is the result if we were to run local search from the above initial state, while only allowing transitions to neighboring states with a lower h value? Suppose we additionally allow transitions to neighbors with *equal* h value. Trace a hill-descent outcome (as in part 2) that results in a consistent solution in the fewest number of iterations.

[No Submission] Problem 3: Constraint Satisfaction

We are trying to schedule five activities (A, B, C, D, E) into four timeslots (1, 2, 3, 4). Each activity may be assigned to any of the timeslots, and a timeslot may contain no or multiple activities, but the following constraints must be satisfied:

- $A > D$
- $C > E$
- $A \neq C$
- $C \neq D$
- $B \geq A$
- $D > E$
- $B \neq C$
- $C \neq D + 1$

1. Suppose we run **one round** of arc consistency, processing each variable once following the order A, B, C, D, E. We do not re-add constraints back into the queue after a variable's domain is pruned. Find the resultant domains for each variable.
2. Now continue running arc consistency from the domains that you found above until the algorithm terminates (i.e., run the AC-3 algorithm). Find the resultant domains for each variable.
3. Is arc consistency by itself sufficient to yield a unique solution to the scheduling problem? Briefly explain why or why not.
4. From the reduced domains, find all valid solutions to the scheduling problem.
5. Going back to the arc-consistent, but unsolved, CSP in part 2, make a variable assignment that does not appear in a valid solution. Trace the subsequent constraint propagation steps, and indicate which constraint is ultimately violated, leading to backtracking.

[Graded] Problem 4: Simulated Annealing

Sudoku is a logic-based number placement puzzle. The basic rules are as follows: Given a $n \times n$ grid (where n is a perfect square) and a set of pre-filled clue cells, fill in the remaining cells such that each row, column, and $\sqrt{n} \times \sqrt{n}$ "major" subgrid contains an instance of each number from 1 to n (inclusive). Classic sudoku has a 9×9 grid.

You will implement a sudoku solver using simulated annealing. A state is a 2D NumPy array with n of each of the numbers from 1 to n. We will only consider states in which every major subgrid is already consistent (a reasonable assumption, as this is trivial to assign). A transition occurs by swapping two non-clue numbers within a major subgrid. Finally, we can measure the number of "errors" for a given state by counting the number of missing values in each row and column.

Part 1: Implementation (10 points)

Implement the `simulated_annealing()` function in `sudoku.py`. You should use the provided `successors()` and `num_errors()` functions. In addition to the initial `board` and `clues` indices, `simulated_annealing()` will also take in several arguments relevant to the temperature schedule. Your temperature parameter `T` should be defined as

```
T = startT * (decay**iter)
```

where `iter` is the iteration number (starting from 0).

The search procedure should end if any one of the following conditions occurs: `T < tol`, the current state has no successors, or the current state is a solution (number of errors = 0). When any of these occurs, return the following two values: the current board state, and a list of integers containing the number of errors of each state encountered (including initial state) during search.

Part 2: Analysis (9 points)

You will be analyzing the performance of simulated annealing on 9×9 sudoku puzzles with $c = 40$ clues. You can run `python sudoku.py` with the `-n` and `-c` options set to these values. Without setting any other parameters, the default values of `startT` and `decay` are 100 and 0.5, respectively. You will likely see the solver failing to find a solution on most instances with these parameters.

1. Use the `-d` option to experiment with the `decay` rate of the temperature. Specifically, find a value such that when you set the option `-b 30` (batch of 30 runs), at least half of the instances have a final error of 0. Explain why the new value is able to improve the performance of simulated annealing. Also generate two plots, one showing the error history of an individual search that successfully finds a solution, and one showing a final error histogram of a batch of 30 searches. It may be easiest to add some code to `sudoku.py` to do this.

2. Keeping the `decay` value that you found above, experiment with the `startT` (starting temperature) parameter using the `-s` option. Show three individual search result plots with this value set to 1, 10, and 100. Explain how the different values of `startT` affect the progression of the search.

3. Repeat the above three experiments on 30-batch runs. Show the resultant histograms of each, and again explain how the different values of `startT` affect (or do not affect) the overall performance of the searches.

Code Submission

Please submit the `sudoku.py` file on Pensive.

[Graded] Problem 5: Grid World Path Planning

You will be implementing search algorithms to solve path planning for an agent in an environment with a set of terrain features. The environment is discretized as a grid world surrounded by four impassable walls. Within the walls, the robot can move to one of up to eight adjacent cells from a given cell.

NumPy Occupancy Grid: The world is represented by a 2D NumPy array, with each value representing a cell. Each cell has a type corresponding to one of the following terrain features, along with an associated cost incurred by the robot when passing through that cell:

- A *prairie* cell has a cost of 1. These are the most common cells that the robot encounters.
- A *desert* cell has a cost of 0.5. The robot moves more easily through these cells as there are less flora and fauna to contend with.

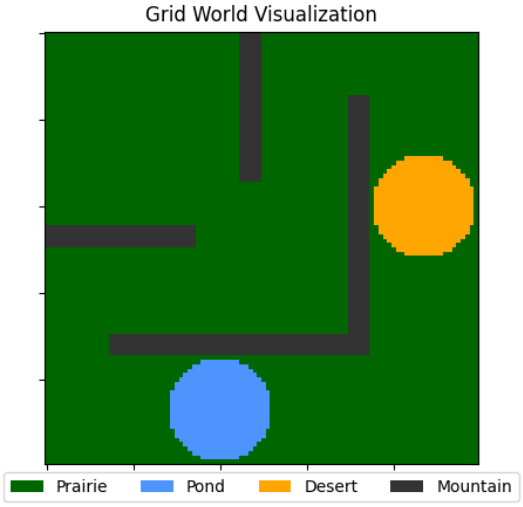


Figure 1: Example grid world with terrain features: prairie, desert, pond, and mountain cells.

- A *pond* cell has a cost of 2. The robot can swim, but it must deploy aquatic gear and move more slowly.
- A *mountain* cell has a cost of ∞. The robot cannot pass through these cells.

Custom Worlds: We have provided four example grid worlds with different arrangements of terrain, all saved as NumPy `.npy` files. You can load an environment using `numpy.load()` and visualize it using `visualize_grid_world()` in the `utils.py` file. An example is shown in Figure 1.

All coding implementations in the following parts will be completed in the `path_finding.py` file.

Part 1: Uninformed Search (10 points)

Recall that depth-first search and breadth-first search do not consider the true costs of the cells in the gridworld. Prairie, desert, and pond cells are all treated the same. The only exception to this rule is that mountain cells are impassable; movement into a mountain cell is not allowed.

Implement the `uninformed_search()` method. The inputs are a `grid`, the `start` and `goal` states (both tuples of grid coordinates), and a `PathPlanMode` variable called `mode`. `mode` may have value `PathPlanMode.DFS` or `PathPlanMode.BFS`, indicating which algorithm to run. For node expansion, use the `expand` method in `utils.py`. You should not add already reached cells to the frontier.

Data structures: You will need to define and update the following variables during the search process. Please adhere to all specs, as some of these will be returned when the method completes.

- `frontier`: This stores the frontier states as the search progresses. You can implement this as a list, and you can remove from either the back or the front of the list depending on the search method. Add successor states to the frontier *in the same order as returned from* `expand()`.
- `frontier_sizes`: This is a list of integers storing the size of the frontier at the beginning of each search iteration (before popping and expanding a node).

- `expanded`: This is a list of all expanded states. You can simply append each state to the list after popping it from the frontier.
- `reached`: This can be implemented as a dictionary. Keys are the states that have been reached (again, these are just tuples of grid coordinates), and corresponding values are their parent states. There is no need to track actions or cost information.

The goal test should be conducted when a node is **popped** from the frontier (**late goal test**). Then you will need to reconstruct the `path` solution. This should be a **list** of coordinate tuples, with `start` and `goal` as the first and last elements, respectively, and a sequence of valid adjacent cells between them. **If the goal was not found, then `path` must be an empty list.** The completed method should return `path`, `expanded`, and `frontier_sizes` (in that order).

Part 1.5: Testing Uninformed Search

You should test your implementation before moving on to the next part. From a terminal, you can run the command `python main.py worlds [id]`, and replace `id` with an integer between 1 and 3 for the sample world that you would like to test on. You should then see a summary of the search results of DFS and BFS in the terminal, as well as a figure pop up with the corresponding visualization. By default, this image is static. You can use the `-a` option to show an animation instead. `-a 1` will animate the expansion process, and `-a 2` will animate the path.

Part 2: Heuristic Computation (2 points)

You will next implement informed search methods, but these will require a heuristic computation. The `compute_heuristic()` method takes a query `node`, the `goal`, as well as the `heuristic` type to use (either `MANHATTAN` or `EUCLIDEAN` distance). It should compute and return an **admissible** heuristic value of the specified type. Implement these computations with an appropriate (but nonzero) *scaling factor* so that admissibility is satisfied.

Part 3: A* Search & Beam Search (10 points)

Now you will implement basic A* search, as well as a small variation of it as beam search. The `a_star()` method takes in the same inputs as `uninformed_search`, in addition to two new ones. `mode` will be either `PathPlanMode.A_STAR` or `PathPlanMode.BEAM_SEARCH`. `heuristic` will be either `Heuristic.MANHATTAN` or `Heuristic.EUCLIDEAN`. `width` will only be used for beam search.

The frontier should now be implemented as a `PriorityQueue` object. To ensure proper sorting of the frontier, each element can be a tuple of the form `(priority, state)`, where `priority` is computed as the sum of the cell's backward cost g and heuristic h. You can use the `cost` method in `utils.py` to compute a cell's cost, and you can use `compute_heuristic()` from above.

The values of the `reached` dictionary must now store a state's parent and cost. A previously reached state may be re-added to the frontier if its new cost is lower than its old one. Finally, if `mode` is `PathPlanMode.BEAM_SEARCH`, the frontier should only keep the `width` cheapest nodes at the end of each iteration. As in `uninformed_search`, your method should return `path`, `expanded`, and `frontier_sizes`. If the goal was not successfully found, `path` should be an empty list.

Part 3.5: Testing A* Search

To test A*, you can again run `python main.py worlds [id]`, and add to this one or two more arguments. With the `-e` option, an argument of 1 will set Manhattan distance, and 2 will set Euclidean distance. This will run both A* and beam search, the latter with a default width of 50. You can also set the `-b` option followed by an integer specifying a different width for beam search.

Part 4: Experiments (9 points)

Once you have finished your implementations, perform the following tasks and provide responses to the questions in the same PDF document as your solutions to the previous problems.

1. Run uninformed search on each of the three worlds. Show the DFS and BFS output figures for one of the worlds (your choice) in your document. What do you notice about the *empirical* time and space complexities of DFS and BFS as indicated by the number of expanded nodes and maximum frontier size? Explain any discrepancies as compared to the *theoretical* complexities of the two algorithms.

2. Run A* and beam search (using the default width) on each of the four worlds. Show the output figures for a world in which the two algorithms return different solutions. Compare the resultant space complexities of the two algorithms, as well as the optimality of the solutions returned in each of the worlds. Also comment on how the choice of heuristic impacts complexity and optimality.

3. Let's take another look at the world for which A* and beam search returned different solutions. Tweak the beam width until you reach a value for which the solutions match (in both path length and cost). Show the output figures, and comment on why this new value is "better" than the default one for finding the solution of A*. How does the complexity of search change in response?

Code Submission

Please submit the `path_finding.py` file on Pensive.